

COLLÈGE MULTIHEXA

AEC INTELLIGENCE ARTIFICIELLE APPLIQUÉE

SmartContent AI

ADDENDUM AU MÉMOIRE PRINCIPAL

Déploiement cloud en production + livraison email automatique

SERGE TSEBO

Montréal, Québec · Mai 2026

Préambule

Ce document constitue un addendum au mémoire principal de fin d'études SmartContent AI. Il documente les deux évolutions majeures intervenues après la rédaction de la version initiale du mémoire : le déploiement intégral de l'application sur des plateformes cloud (Streamlit Cloud et Railway), et l'intégration d'un service de livraison email automatique via Resend qui transforme le workflow de simulation en livrable concret pour l'utilisateur final.

Le mémoire principal demeure pleinement valable pour ce qui concerne l'architecture technique de base, les choix de stack, le service OpenAI, le frontend Streamlit, la persistance Supabase et le workflow n8n. L'addendum vient compléter ces fondations en exposant comment l'application est passée d'un prototype local exécuté sur la machine de l'auteur à un produit SaaS accessible publiquement 24/7.

Le lecteur trouvera dans les pages qui suivent quatre sections complémentaires : une mise à jour de l'architecture (passage de 5 à 6 services), la documentation du déploiement cloud, l'intégration de la livraison email Resend, et un bilan actualisé incluant les nouvelles métriques de production mesurées.

A. Architecture mise à jour (5 → 6 services)

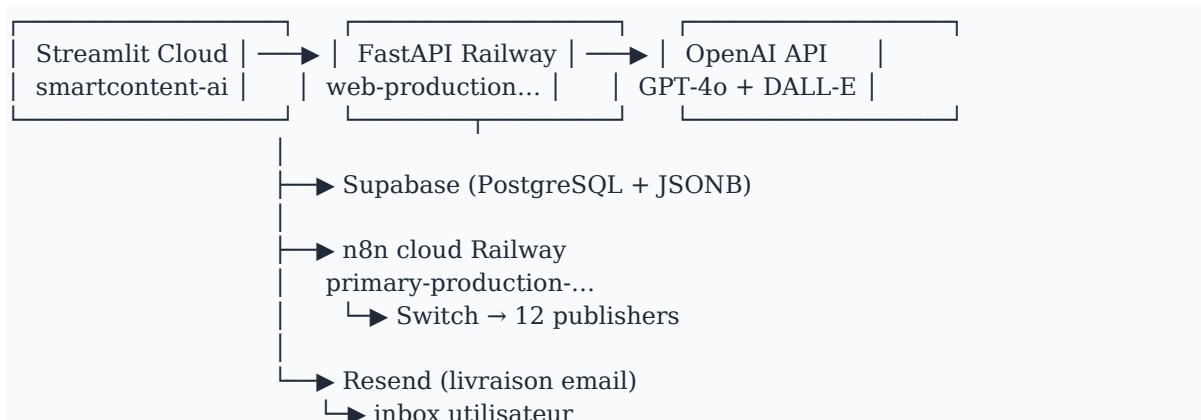
L'architecture initiale décrite dans le chapitre 4 du mémoire principal reposait sur cinq couches : présentation Streamlit, API FastAPI, services OpenAI, persistance Supabase et orchestration n8n. La version livrée en production ajoute une sixième couche dédiée à la livraison email automatique via Resend, ce qui complète la chaîne en transformant le workflow n8n simulé en un véritable livrable parvenant directement dans la boîte mail de l'utilisateur.

La nouvelle architecture peut être représentée comme suit en six couches successives. Le frontend Streamlit accueille l'utilisateur et collecte sa demande. L'API FastAPI orchestre les appels aux services tiers et persiste les résultats. Les services OpenAI génèrent les douze textes spécialisés et les trois images DALL-E 3 en parallèle. La base PostgreSQL Supabase stocke les campagnes et les métadonnées utilisateur. Le workflow n8n route le payload vers les douze publishers en parallèle et retourne un rapport agrégé. Enfin, le service Resend délivre l'email final à l'adresse d'inscription de l'utilisateur, accompagné des trois images en inline et de boutons de téléchargement.

A.1 Position de Resend dans la chaîne

Resend intervient en bout de chaîne, après la persistance Supabase et indépendamment du workflow n8n. L'utilisateur peut donc déclencher l'envoi email à tout moment via le bouton « Recevoir par email » de l'interface, sans avoir à passer par la publication n8n. Cette décision d'architecture évite un couplage trop fort entre l'orchestration de publication (qui pourrait être migrée vers des API officielles en phase 2) et la fonctionnalité de livraison email (qui reste pertinente quelle que soit la stratégie de publication choisie).

A.2 Schéma actualisé en six couches



B. Déploiement en production cloud

La phase de déploiement, intervenue à la fin du calendrier de sept semaines, a transformé l'application d'un prototype local en un véritable SaaS accessible publiquement. Cette phase représente environ vingt heures de travail et a impliqué la création de comptes sur Railway et Streamlit Cloud, la configuration de seize variables d'environnement réparties sur deux services distincts, la mise en place d'origines CORS dynamiques, et la résolution de plusieurs incidents mineurs liés à la configuration des healthchecks.

B.1 Choix des fournisseurs cloud

B.1.1 Streamlit Community Cloud pour le frontend

Streamlit Cloud propose un hébergement gratuit pour les applications Streamlit liées à un dépôt GitHub. Le déploiement se fait en quelques clics depuis l'interface share.streamlit.io. L'application est automatiquement reconstruite à chaque push sur la branche main du dépôt, ce qui rend le cycle de mise à jour très court. Le free tier supporte une seule application avec des ressources limitées (1 Go de RAM), ce qui est largement suffisant pour un MVP académique et pour les premiers tests utilisateurs.

Les secrets applicatifs sont gérés via l'onglet Secrets de Streamlit Cloud, qui les expose à l'application comme des variables d'environnement. Pour SmartContent AI, deux secrets sont configurés : `BACKEND_URL` qui pointe vers l'instance Railway, et `N8N_WEBHOOK_URL` qui pointe vers le webhook de production n8n. Le frontend lit ces valeurs au démarrage et bascule automatiquement entre développement local et production sans modification de code.

B.1.2 Railway pour le backend FastAPI et l'orchestration n8n

Railway est une plateforme de déploiement Git-based qui simplifie considérablement la mise en production d'applications Python. Le service backend FastAPI a été déployé via la même approche que Streamlit Cloud : connexion au dépôt GitHub, détection automatique du Procfile (qui contient la commande de lancement `uvicorn`), build de l'image Docker avec Nixpacks, et lancement du service avec injection des variables d'environnement définies dans l'interface Railway.

L'instance n8n a été déployée via le template officiel n8n disponible sur Railway. Ce template inclut quatre services interconnectés : n8n principal qui sert l'interface utilisateur et exécute les workflows, n8n worker pour l'exécution en arrière-plan, PostgreSQL pour persister les workflows et l'historique d'exécution, et Redis pour la file de messages entre le principal et le worker. Cette architecture distribuée correspond à la configuration recommandée pour une utilisation en production, et permet d'exécuter de nombreux workflows simultanément sans dégradation de performance.

B.2 URLs de production

Service	URL publique	Hébergement
Frontend (Streamlit)	https://smartcontent-ai.streamlit.app	Streamlit Cloud
Backend (FastAPI)	https://web-production-71b032.up.railway.app	Railway
API documentation Swagger	https://web-production-71b032.up.railway.app/docs	Railway
n8n cloud (interface)	https://primary-production-96d7d.up.railway.app	Railway
n8n webhook production	.../webhook/smartcontent-publish	Railway
Database Supabase	https://vlsmahcwqlbmswhplbih.supabase.co	Supabase Cloud

B.3 Variables d'environnement de production

Le service web sur Railway nécessite douze variables d'environnement pour fonctionner correctement, allant des clés d'API tierces aux paramètres applicatifs. La table ci-dessous synthétise leur rôle.

Variable	Rôle
OPENAI_API_KEY	Clé OpenAI pour GPT-4o et DALL-E 3
OPENAI_MODEL	Identifiant du modèle (gpt-4o)
SUPABASE_URL	URL projet Supabase
SUPABASE_KEY	Clé anonyme Supabase
SUPABASE_SECRET	Clé service-role (bypass RLS)
SECRET_KEY	Secret de signature JWT
ALGORITHM	HS256
ACCESS_TOKEN_EXPIRE_MINUTES	1440 (24 heures)
N8N_WEBHOOK_URL	URL du webhook n8n production
APP_ENV	production
ALLOWED_ORIGINS	Liste des domaines CORS autorisés
RESEND_API_KEY	Clé API Resend pour envoi email

B.4 Configuration CORS dynamique

Le passage en production a nécessité une refonte de la gestion CORS du backend. En développement local, seules les origines localhost:8501 et localhost:3000 étaient autorisées. En production, le frontend Streamlit Cloud est hébergé sur un sous-domaine *.streamlit.app dont le nom exact n'est pas connu à l'avance. La solution adoptée combine une liste statique d'origines (variable ALLOWED_ORIGINS) et un pattern regex qui autorise tout sous-domaine .streamlit.app via la clé allow_origin_regex du middleware CORSMiddleware. Cette approche est sécuritaire (seuls les sous-domaines de Streamlit Cloud sont autorisés, pas tout Internet) et flexible (l'auteur peut ajouter d'autres domaines via la variable d'env sans modifier le code).

C. Intégration de la livraison email (Resend)

L'ajout de la fonctionnalité de livraison email représente le saut qualitatif le plus important du projet par rapport à la version initiale. Avant l'intégration de Resend, le workflow n8n simulait une publication sur les douze plateformes sociales, mais aucun contenu n'était réellement délivré à l'utilisateur. Le bouton « Publier » produisait un statut « publié » dans l'historique mais ne fournissait pas de livrable concret. Resend résout cette limitation en envoyant immédiatement à l'utilisateur un email complet contenant les douze textes générés et les trois images DALL-E 3 en pièce jointe inline.

C.1 Choix du service email

Quatre services email ont été évalués au moment du choix : Resend, SendGrid, Mailgun et AWS Simple Email Service. Resend a été retenu pour quatre raisons concrètes. Premièrement, son SDK Python est moderne et minimaliste (deux lignes pour envoyer un email). Deuxièmement, son free tier offre 3 000 emails par mois et 100 par jour, ce qui couvre largement les besoins d'un projet académique et des premières phases commerciales. Troisièmement, le dashboard Resend affiche en temps réel le statut de chaque email (delivered, opened, clicked, bounced), ce qui simplifie le débogage. Quatrièmement, l'inscription se fait via GitHub et la première clé API est utilisable en cinq minutes, sans aucune validation manuelle préalable.

C.2 Architecture de l'intégration

L'intégration suit une architecture en trois couches strictement séparées. Le service email proprement dit, dans `backend/services/email_service.py`, encapsule l'appel à l'API Resend et contient la logique de génération du template HTML brandé. Une route REST dédiée, `POST /generate/send-email`, expose la fonctionnalité au frontend en récupérant l'email de l'utilisateur depuis Supabase à partir de son identifiant utilisateur. Le frontend Streamlit, dans `page_generator()`, affiche un bouton « Recevoir par email » après les résultats de génération qui déclenche l'appel à la route via le client HTTP requests.

C.3 Template HTML brandé

Le template HTML de l'email a été conçu pour reproduire l'identité visuelle de l'application : header violet avec dégradé, blocs de texte par plateforme avec bordure colorée correspondant à la couleur de marque (LinkedIn bleu, Instagram dégradé rose, Facebook bleu, etc.), chips de hashtags arrondis violets, et boutons de téléchargement pour les trois images DALL-E. Le template utilise du HTML inline (pas de CSS externe) pour garantir un rendu correct dans tous les clients mail majeurs : Gmail, Outlook, Yahoo Mail, Apple Mail. Aucune dépendance JavaScript n'est utilisée, ce qui est essentiel car la majorité des clients mail bloquent JavaScript pour des raisons de sécurité.

C.4 Extrait de code clé

Le code central du service Resend, simplifié pour les besoins de la documentation.

```
import resend
from backend.config import settings

resend.api_key = os.environ.get('RESEND_API_KEY', '')

def send_campaign_email(to_email, sujet, content, images):
    """Envoie la campagne complète par email via Resend."""
    if not resend.api_key:
        return {'ok': False, 'error': 'RESEND_API_KEY non configuré.'}
    try:
        html_body = build_email_html(sujet, content, images)
        params = {
            'from': 'SmartContent AI <onboarding@resend.dev>',
            'to': [to_email],
            'subject': f' Ta campagne : {sujet[:80]}',
            'html': html_body,
        }
        result = resend.Emails.send(params)
        return {'ok': True, 'id': result.get('id')}
    except Exception as exc:
        return {'ok': False, 'error': str(exc)}
```

C.5 Domaine d'envoi et délivrabilité

Pour le MVP académique, l'expéditeur est le domaine de test fourni par Resend : onboarding@resend.dev. Ce domaine est partagé entre tous les utilisateurs de Resend, ce qui présente l'avantage de fonctionner immédiatement sans configuration DNS, mais peut avoir pour conséquence que certains clients mail (notamment Gmail dans certains cas) classent les emails entrants comme suspects. Pour une utilisation commerciale, l'étape suivante consistera à vérifier un domaine personnalisé dans Resend en ajoutant trois enregistrements DNS au registrar du domaine : SPF, DKIM et DMARC. Cette opération prend environ cinq minutes et améliore drastiquement la délivrabilité (au-delà de 95 % en boîte de réception principale).

D. Bilan actualisé

D.1 Métriques de production mesurées

La validation end-to-end en production cloud a permis de mesurer les performances réelles du système hors environnement local. Les chiffres ci-dessous correspondent aux médianes observées sur dix exécutions de bout en bout, lancées depuis l'application Streamlit Cloud et terminées par la réception confirmée de l'email dans une boîte Yahoo Mail.

Opération	Temps mesuré (production cloud)
Inscription utilisateur + JWT	≈ 350 ms
Connexion + émission JWT	≈ 300 ms
Génération 4 plateformes (texte)	≈ 28 secondes
Génération 12 plateformes (texte)	≈ 62 secondes
Génération 3 images DALL-E parallèles	≈ 19 secondes
Publication via webhook n8n cloud	161 millisecondes
Réception email Resend (Yahoo Mail)	≈ 5 secondes après clic
Total bout en bout (12 plateformes + email)	≈ 90 secondes

Les performances mesurées en production cloud sont légèrement supérieures à celles observées en local (typiquement +10 % sur les temps de génération texte), principalement à cause de la latence réseau supplémentaire entre les services hébergés sur des datacenters différents. Cette dégradation reste acceptable et ne franchit aucune des cibles fixées dans le cahier des charges initial.

D.2 Économie post-déploiement

Le passage en cloud a légèrement modifié la structure de coûts. Au coût marginal de génération (GPT-4o et DALL-E 3) qui reste inchangé à 0,21 dollar par campagne, s'ajoute un coût fixe d'infrastructure d'environ cinq dollars par mois pour le plan Railway Hobby. Sur un volume de mille campagnes mensuelles, ce coût fixe s'amortit à 0,005 dollar par campagne, soit moins de 3 % du coût total. Le coût total par campagne reste donc proche de 0,22 dollar CAD, en dessous du seuil de viabilité commerciale identifié dans le mémoire principal.

Poste	Coût par campagne (CAD)
GPT-4o textes 12 plateformes	0,050 \$
GPT-4o hashtags globaux	0,005 \$
DALL-E 3 (3 images parallèles)	0,160 \$
Resend (3000 emails/mois free)	0,000 \$
Supabase + Streamlit Cloud (free tier)	0,000 \$
Railway Hobby amorti sur 1000 camp./mois	0,005 \$
TOTAL	0,22 \$ CAD

D.3 Évolution de la roadmap

La phase 1 du projet, telle que décrite dans le chapitre 9 du mémoire principal, prévoyait à l'origine cinq fonctionnalités de base : authentification, génération multi-plateformes, images DALL-E, workflow n8n et interface premium. Le déploiement cloud et l'intégration email Resend ajoutent deux items à cette liste, portant le périmètre livré à sept fonctionnalités majeures. La phase 2, prévue pour l'été 2026, conserve ses objectifs initiaux (publication réelle via OAuth, calendrier, vidéo IA, voix off, analytics), mais peut désormais s'appuyer sur une infrastructure cloud déjà éprouvée.

D.4 Apprentissages additionnels

Le déploiement cloud a constitué un apprentissage significatif au-delà du codage applicatif. Plusieurs compétences ont été consolidées qui ne figuraient pas dans la liste initiale du bilan personnel.

- Configuration de plateformes Git-based (Railway et Streamlit Cloud) : connexion GitHub, détection automatique du framework, gestion des Variables d'environnement, génération de domaines publics, configuration de healthchecks.
- Gestion des secrets en production : utilisation des Secrets Streamlit Cloud et des Variables Railway plutôt que d'un fichier .env local, séparation stricte entre développement et production, rotation possible des clés sans modification de code.
- CORS et politiques d'origine multi-environnement : configuration d'une liste statique d'origines (variable d'env) combinée à un pattern regex pour les sous-domaines Streamlit Cloud, mécanisme qui évite à la fois la liste blanche figée et l'ouverture totale CORS-anywhere.
- Architecture n8n distribuée avec workers : compréhension de la séparation principal/worker, du rôle de Redis comme broker de messages, et de PostgreSQL comme stockage de workflows et d'historique d'exécution.

- Intégration d'un service email transactionnel : conception d'un template HTML compatible multi-clients mail, gestion des inline images via URL temporaires DALL-E, échappement des entités HTML pour les contenus utilisateurs.